brain/carl_stdp.py — Full Line-by-Line Walkthrough

Length: 278 lines. **Role:** three related but distinct pieces — a general-purpose reward-modulated STDP synapse population (`STDPSynapse`), a higher-level wrapper that uses that synapse to score candidate actions (`STDPActionEvaluator`), and an unrelated "quantum-inspired" probabilistic decision-maker (`QuantumDeliberator`). Like `carl_omega_extensions.py`, none of this file is imported by `carl_simulation.py` — CARL's actual live obstacle-avoidance learning happens through the separate, simpler R-STDP rule built directly into `brain/carl_reflex.py`. This file is a more general, standalone STDP toolkit belonging to the same alternate ("GENESIS scout") architecture as `carl_omega_extensions.py` and `carl_physarum.py`, built around an action-selection scheme (`action_map`, a discrete list of candidate control values) that has no counterpart in the live reservoir/reflex control loop.

Lines 1–16: Header docstring

```python
"""
carl_stdp.py — Reward-Modulated Spike-Timing Dependent Plasticity (R-STDP)
Based on: Markram 1997 (STDP) + Izhikevich 2007 (reward modulation + eligibil

Three-factor learning rule:
  ΔW = learning_rate × Reward × Eligibility_Trace

Where eligibility trace tracks STDP correlations:
  dE/dt = -E/tau_E + STDP(pre, post)
...
"""
```

States the governing equation up front, and it's genuinely more general than the R-STDP already seen in `carl_reflex.py`: that file hard-wired STDP learning directly onto a 24-sensor → 2-motor-neuron obstacle-avoidance circuit. This file instead builds STDP as a reusable class that can connect *any* pre-synaptic population to *any* post-synaptic population — the "three-factor rule" name refers to the fact that a weight change here depends on three separate signals (pre-activity, post-activity, and a delayed reward), rather than the two-factor "cells that fire together wire together" of classic unsupervised Hebbian learning, which the docstring explicitly calls out as less precise ("far more precise than Hebb's 'fire together wire together'").

Lines 21–29: Global hyperparameters

```python
TAU_PRE   = 20.0   # ms — pre-synaptic trace decay
TAU_POST  = 20.0   # ms — post-synaptic trace decay
TAU_E     = 50.0   # ms — eligibility trace decay
A_PLUS    = 0.01   # LTP amplitude
A_MINUS   = 0.012  # LTD amplitude (slightly asymmetric — bias toward depress
W_MAX     = 2.0    # Maximum synaptic weight
W_MIN     = -0.5   # Minimum (allows weak inhibition)
LR        = 0.008  # Reward modulation learning rate
```

`TAU_PRE`/`TAU_POST` (20ms) are shorter than `TAU_E` (50ms) — the individual pre/post activity traces fade quickly, but the *combined* eligibility trace (which STDP correlations feed into) persists roughly 2.5× longer, giving the delayed reward signal a wider window to actually "catch" and reinforce a recent correlation, which is the entire point of an eligibility trace mechanism: bridging the gap between an action and a reward that arrives some time later. `A_MINUS > A_PLUS` (0.012 vs 0.01) is a deliberate, named asymmetry — the comment states it directly: the learning rule is biased slightly toward weakening (depression) over strengthening (potentiation), a common real design choice in STDP models to keep overall network activity from runaway growth. `W_MIN = -0.5` (allowing weak inhibitory/negative weights) alongside `W_MAX = 2.0` (a much larger positive ceiling) is an asymmetric weight range — connections can become somewhat suppressive but can become much more strongly excitatory, reflecting a plausible design assumption that this synapse population represents mostly excitatory pathways with limited inhibitory capacity.

`STDPSynapse` (lines 32–120)

Lines 40–60: `__init__`

```python
```

```
def __init__(self, pre_size, post_size, init_scale=0.1):
    self.pre_size  = pre_size
    self.post_size = post_size
    self.W = np.random.randn(pre_size, post_size) * init_scale
    self.E = np.zeros((pre_size, post_size), dtype=float)
    self.A_pre  = np.zeros(pre_size,  dtype=float)
    self.A_post = np.zeros(post_size, dtype=float)
    self.reward_buf = deque(maxlen=5)
    self.ltp_events = 0
    self.ltd_events = 0
    self.total_weight_change = 0.0
```

`W` is a full dense `pre_size × post_size` weight matrix, randomly initialized with small values (std `0.1`) — every pre-synaptic unit starts with some connection to every post-synaptic unit, and learning will sculpt those into something meaningful. `E` (eligibility) is the same shape as `W` — one eligibility value tracked per individual synapse. `A_pre`/`A_post` are the decaying per-neuron activity traces described in the header equations. `reward_buf` is a short 5-entry rolling window of recent reward values — the "three-factor rule needs short delay" comment explains why: rather than reacting to a single noisy instantaneous reward, the rule averages over the last few, smoothing out momentary spikes. The three counters (`ltp_events`, `ltd_events`, `total_weight_change`) exist purely for diagnostics, surfaced later by `plasticity_report()`.

Lines 62–108: `step()` — the actual learning update

python

```
decay_pre  = np.exp(-dt / TAU_PRE)
decay_post = np.exp(-dt / TAU_POST)
decay_e    = np.exp(-dt / TAU_E)

self.A_pre  *= decay_pre
self.A_post *= decay_post
self.E      *= decay_e
```

Exact-form exponential decay (the same `exp(-dt/tau)` pattern used for the more numerically precise decays in `carl_reflex.py`, as opposed to the reservoir's simpler linear-approximation Euler decay) applied to all three trace types before anything else happens this step.

python

```
if np.any(post_spikes > 0):
    ltp = np.outer(self.A_pre, post_spikes) * A_PLUS
    self.E  += ltp
    self.ltp_events += int(np.sum(post_spikes > 0))
```

LTP ("Long-Term Potentiation," synaptic strengthening) branch: if any post-synaptic unit just spiked, `np.outer(A_pre, post_spikes)` builds a full `pre × post` matrix where entry `[i,j]` is nonzero only where post-unit `j` just spiked, scaled by how recently/strongly pre-unit `i` had spiked (its lingering trace `A_pre[i]`). This is the "pre-before-post strengthens" half of STDP: a pre-synaptic spike that recently occurred, followed now by a post-synaptic spike, is exactly the causal "pre helped cause post" pattern that should be reinforced. It's added into the eligibility trace, not directly into the weights — the actual weight change waits for the reward signal further down.

python

```
if np.any(pre_spikes > 0):
    ltd = np.outer(pre_spikes, self.A_post) * (-A_MINUS)
    self.E  += ltd
    self.ltd_events += int(np.sum(pre_spikes > 0))
```

LTD ("Long-Term Depression," weakening) branch: the mirror-image case — a post-synaptic unit that recently fired (its trace `A_post[j]` still elevated), followed now by a pre-synaptic spike, is the "post-before-pre" (backward/acausal) timing pattern that classic STDP theory says should weaken that connection — hence the negated `-A_MINUS` scale.

python

```
self.A_pre  += pre_spikes
self.A_post += post_spikes
```

The comment above this in the source ("Update traces AFTER correlation (causal order)") flags a subtlety worth internalizing: the LTP/LTD comparisons just above deliberately use the *pre-update* trace values (i.e., activity from *before* this tick's own spikes), so a spike happening on this exact tick doesn't get credit for "causing" a same-tick spike in the other population — only genuinely prior activity (already baked into the decayed trace) counts toward this tick's eligibility change. Only after that comparison is done do the current tick's own spikes get folded into the traces, ready to influence *future* comparisons.

python

```
        self.reward_buf.append(reward)
        avg_reward = float(np.mean(self.reward_buf)) if self.reward_buf else 0.0

        if abs(avg_reward) > 1e-4:
            dW = LR * avg_reward * self.E
            self.W  = np.clip(self.W + dW, W_MIN, W_MAX)
            self.total_weight_change += float(np.linalg.norm(dW))

        return self.W.copy()
```

This is the "third factor": the actual weight change (`dW`) is the eligibility trace `E` (which by itself records only *timing correlation*, with no notion of good or bad) multiplied by the recent average reward. A connection with high eligibility but zero/near-zero reward doesn't change at all — eligibility alone is not enough, exactly matching the three-factor framing in the header docstring. If the average reward is meaningfully positive, all eligible connections get strengthened *in the direction their eligibility indicates*; if reward is negative, the same eligible connections get pushed the opposite way. `np.clip` enforces the `[W_MIN, W_MAX]` bounds set as global constants. `.copy()` on the return value protects the caller from accidentally holding a live, mutable reference into this object's internal weight matrix.

Lines 110–112: `predict()`

```python
def predict(self, pre_activity):
    return np.tanh(self.W.T @ pre_activity)
```

A simple forward pass: given a pre-synaptic activity vector, project it through the (transposed) learned weight matrix and squash with `tanh` to bound the output — this is what lets the learned synapse be *used* for something (predicting/scoring post-synaptic activity) rather than only ever being trained.

Lines 114–120: `plasticity_report()`

```python
def plasticity_report(self):
    return {
        'ltp': self.ltp_events,
        'ltd': self.ltd_events,
        'mean_w': float(np.mean(self.W)),
        'total_dw': round(self.total_weight_change, 4),
    }
```

A read-only diagnostics snapshot, presumably intended for logging/telemetry in whatever larger system eventually uses this class.

`STDPActionEvaluator` (lines 124–203) — turning STDP into an action-quality scorer

Lines 136–150: `__init__`

```python
def __init__(self, state_dim=5, visual_dim=15, n_actions=9):
    self.state_dim = state_dim
    self.visual_dim = visual_dim
    self.n_actions = n_actions
    self.full_dim = state_dim + visual_dim
    self.synapse = STDPSynapse(self.full_dim, n_actions, init_scale=0.05)
    self.reward_baseline = 0.0
    self.baseline_tau    = 200.0
    self.action_map = [-8., -5., -2., -1., 0., 1., 2., 5., 8.]
```

The comment describes this class's purpose plainly: "Replaces the hand-crafted Hebbian associator with a proper R-STDP network." The pre-synaptic population here is a combined `state_dim + visual_dim = 20`-element representation (proprioceptive state plus some 15-dimensional visual encoding, `v_pop` — from a different sensing pipeline than the SSD-Lite vision model used in the live simulation). The post-synaptic population is a set of `n_actions=9` discrete candidate actions, listed explicitly in `action_map` — nine specific control values from `-8.0` to `+8.0`. This confirms the alternate architecture this file belongs to picks actions from a small discrete menu rather than the reservoir's continuous 3-value output — a genuinely different control paradigm from the live simulation's approach.

Lines 152–155: `encode_state()`

```python
def encode_state(self, x, v_pop):
    proprio = np.clip(np.abs(x) / np.array([8., 3., 0.7, 3., 0.35]), 0., 1.)
    return np.concatenate([proprio, v_pop])
```

Converts a raw 5-dimensional proprioceptive state vector into firing rates in `[0,1]` by dividing each component by its own typical operating range (`8`, `3`, `0.7`, `3`, `0.35` — presumably position/velocity/pitch/pitch-velocity/neck-angle scale factors matching the same 5-state convention seen in `carl_omega_extensions.py` and `world/carl_mj_physics.py`'s `get_state()`), taking the absolute value (direction/sign

information is discarded in this rate-coding scheme — only magnitude matters for firing rate), and clamping to the valid `[0,1]` range. Then it's concatenated with the separately-supplied visual population vector to form the full 20-dimensional pre-synaptic input.

Lines 157–162: `encode_action()`

```python
def encode_action(self, u):
    idx = int(np.argmin([abs(u - a) for a in self.action_map]))
    spikes = np.zeros(self.n_actions)
    spikes[idx] = 1.0
    return spikes
```

Given a (possibly continuous) chosen control value `u`, this finds the closest entry in the discrete `action_map` and one-hot-encodes it — turning "the action taken" into a spike pattern where exactly one of the nine post-synaptic action units fires.

Lines 164–185: `update()` — the actual learning call

```python
def update(self, xk, v_pop, uk, surprise, nm_DA):
    pre_spikes  = self.encode_state(xk, v_pop)
    post_spikes = self.encode_action(uk)

    raw_reward  = nm_DA - surprise * 0.5
    reward = raw_reward - self.reward_baseline
    self.reward_baseline += (raw_reward - self.reward_baseline) / self.baseli

    threshold = 0.15
    pre_spikes  = pre_spikes * float(surprise > threshold)
    post_spikes = post_spikes * float(surprise > threshold)

    self.synapse.step(pre_spikes, post_spikes, reward, dt=1.0)
```

`raw_reward = nm_DA - surprise * 0.5` defines the reward signal fed into learning as "high dopamine, low surprise" — a state that combines feeling good (elevated DA) with the outcome having matched expectations (low RLS-style prediction error), which is a coherent, biologically-motivated composite reward. Subtracting `reward_baseline` (a slowly-updating running average, with a long `baseline_tau=200` step time constant) turns the raw reward into a genuine reward-prediction-error-like signal — rewarding *better than typical recently*, not just "reward was positive," the same design principle behind `get_reward_prediction_error()` in `carl_endocrine.py`. The baseline itself

updates with a simple exponential moving-average step. The threshold check (`surprise > 0.15`) is a gating mechanism: pre/post spikes are only allowed to register at all — literally zeroed out otherwise — when something sufficiently surprising happened. This means routine, unsurprising ticks contribute *nothing* to learning at all (both spike vectors become all-zero, so `STDPSynapse.step()`'s `if np.any(...)` checks skip the LTP/LTD branches entirely), concentrating all plasticity on genuinely novel or unexpected moments rather than diluting it across every single tick.

Lines 187–200: `action_quality()` and `best_action_bias()`

```python
def action_quality(self, x, v_pop, u):
    pre = self.encode_state(x, v_pop)
    q   = self.synapse.predict(pre)
    idx = int(np.argmin([abs(u - a) for a in self.action_map]))
    return float(-q[idx])

def best_action_bias(self, x, v_pop):
    pre = self.encode_state(x, v_pop)
    return self.synapse.predict(pre)
```

`action_quality()` scores one specific candidate action `u` in the given state, by running the learned synapse's `predict()` forward pass and reading off the entry for whichever discrete action `u` maps closest to. The `-q[idx]` negation is explained directly in the comment: the synapse's raw output represents learned "quality" (higher = better action), but this method is meant to plug into a cost-minimizing planner elsewhere ("Used as a prior in action selection (replaces Hebbian association_cost)"), so it's flipped into a cost (higher quality → lower cost). `best_action_bias()` is the same computation without picking out a single action — it returns the full 9-element quality vector for every candidate action at once, useful for a planner that wants to compare all options directly.

Lines 202–203: `plasticity_stats()`

```python
def plasticity_stats(self):
    return self.synapse.plasticity_report()
```

Thin pass-through to the underlying synapse's diagnostics.

`QuantumDeliberator` (lines 207–275) — probabilistic action selection via wave interference

Lines 206–220: Header and `__init__`

```python
class QuantumDeliberator:
    """
    Innovation 6: Quantum-inspired decision making.
    Instead of argmin(cost), each strategy is a quantum amplitude.
    Interference between strategies: aligned predictions amplify,
    conflicting predictions cancel. Final choice by Born rule probability.
    """
    def __init__(self):
        self.history = deque(maxlen=50)
```

This class is unrelated to STDP — it shares the file only because it's another candidate-action-selection mechanism for the same alternate architecture. The stated goal is a genuinely probabilistic (not merely noisy) decision process, borrowing the mathematical formalism of quantum mechanics (complex amplitudes, interference, and the Born rule for converting amplitudes into probabilities) as a *metaphor and toolset* for structured decision uncertainty — not an actual physical quantum computation.

Lines 222–264: `deliberate()`

```python
def deliberate(self, strategies_costs):
    if not strategies_costs:
        return ('BALANCED', 0., 0.)
    costs = np.array([c for _, _, c in strategies_costs], dtype=float)
```

Takes a list of `(name, u, s, cost)` tuples — named candidate strategies, each with an associated scalar cost (presumably from some planner elsewhere) — and falls back to a default `'BALANCED'` no-op choice if the list is empty.

```python
    costs_norm = costs - costs.min()
    if costs_norm.max() > 1e-6:
        costs_norm /= costs_norm.max()
    phases = costs_norm * math.pi
    amplitudes = np.exp(1j * phases)
```

Normalizes costs into `[0,1]` (lowest-cost strategy becomes 0, highest becomes 1), then maps that onto a phase angle between `0` and `π` radians. Each strategy becomes a complex number on the unit circle (`e^{iφ}`, using NumPy's `1j` imaginary unit) — low-cost strategies get a small phase (close to the real axis, "constructive" per the comment),

high-cost strategies get a phase near `π` (pointing the opposite direction on the complex plane, "destructive").

```python
    total_amplitude = amplitudes.sum()
    coherent_probs = np.abs(amplitudes * np.conj(total_amplitude)).real
    coherent_probs = np.clip(coherent_probs, 0., None)
    total_p = coherent_probs.sum()
    if total_p < 1e-8:
        probs = np.ones(len(strategies_costs)) / len(strategies_costs)
    else:
        probs = coherent_probs / total_p
```

`amplitudes.sum()` is the coherent (interfering) sum of every strategy's complex amplitude — this is where "interference" actually happens: strategies whose phases are close together add constructively (their sum has larger magnitude), while strategies with sharply differing phases partially cancel. `amplitudes * np.conj(total_amplitude)` measures each individual strategy's *overlap* with that combined total direction — a strategy well-aligned with the group consensus gets a larger overlap magnitude than one that's an outlier. Taking `.real` after this multiplication and clipping negative values to zero converts these overlap scores into legitimate (non-negative) unnormalized probabilities, which are then divided by their sum to become a proper probability distribution (`probs`) — a fallback to uniform-random probabilities handles the degenerate case where all overlaps come out essentially zero.

```python
    chosen_idx = int(np.random.choice(len(strategies_costs), p=probs))
    name, u, s, _ = strategies_costs[chosen_idx]
    self.history.append({'probs': probs.copy(), 'chosen': chosen_idx})
    return name, u, s, probs
```

Samples one strategy according to the computed probability distribution — the "Born rule" reference in the docstring is the real quantum-mechanics principle that measurement outcome probabilities equal squared amplitude magnitudes; this code applies an analogous (though not identical — it uses `Re(A·conj(total))` rather than `|A|²`) construction to turn interference patterns into sampling probabilities. The chosen strategy and the full probability distribution are both returned, and the distribution is logged into `history` for the uncertainty calculation below.

Lines 266–274: `decision_uncertainty()`

```python
```

```
def decision_uncertainty(self):
    if not self.history:
        return 0.5
    last = self.history[-1]['probs']
    last = np.clip(last, 1e-8, 1.)
    entropy = float(-np.sum(last * np.log(last)))
    max_ent = float(np.log(len(last)))
    return float(entropy / max_ent) if max_ent > 0 else 0.5
```

Computes Shannon entropy of the most recent decision's probability distribution (`-Σ p·log(p)`, the standard information-theoretic formula for "how spread out/uncertain is this distribution") and normalizes it by the maximum possible entropy for that many options (`log(n)`, the entropy of a perfectly uniform distribution) — producing a clean 0-to-1 "genuine decision uncertainty" score: near 0 means one strategy dominated the probability mass (a confident choice), near 1 means probability was spread almost evenly across all strategies (a genuinely ambiguous, close-call decision). `np.clip(last, 1e-8, ...)` guards against `log(0)`, which would otherwise produce `-inf` for any strategy assigned exactly zero probability.

Line 277: A trailing, oddly-placed import

```
python

import math
```

This sits after every class definition, at the very bottom of the file, rather than grouped with the other imports at the top (line 5 only imports `numpy` and `collections.deque`, not `math`, even though `QuantumDeliberator.deliberate()` uses `math.pi` on line 239 — well before this import statement appears in the file). This works correctly in Python because import statements execute once when the *module* is first loaded, and the whole module (including this bottom-of-file import) finishes loading before any class's methods are ever called — by the time `deliberate()` actually runs, `math` has long since been imported into the module's namespace. It's unconventional placement (Python style conventions strongly favor grouping all imports at the very top of a file) rather than a functional bug, but worth knowing if you're reading top-to-bottom and wondering why `math.pi` seems to reference an undefined name partway through the file.

Summary

A general-purpose, three-factor R-STDP toolkit (`STDPSynapse` + `STDPActionEvaluator`) built around a discrete 9-action menu, plus an unrelated but genuinely interesting probabilistic decision-maker (`QuantumDeliberator`) that borrows quantum-interference math to produce structured, non-uniform action-selection uncertainty rather than plain

random noise. Both pieces are complete and self-consistent, but neither is currently wired into `carl_simulation.py`'s live 30 Hz loop — CARL's actual running obstacle-avoidance learning uses the simpler, purpose-built R-STDP rule inside `carl_reflex.py` instead.